

Practica 4

PMA

Ejercicio 1

a_

```
chan colaBanco(int);
chan respuestas[N](boolean);

proccess Cliente [id=0 to n-1] {
    boolean ok;
    send colaBanco(id);
    receive respuestas[id](ok);
}

proccess Empleado {
    int idC;
    for (int i=0; i<n; i++) {
        receive colaBanco(idC);
        // atendiendo cliente
        send respuestas[idC](true);
    }
}
```

b_

```
chan colaBanco(int);
chan respuestas[N](boolean);

proccess Cliente [id=0 to n-1] {
    boolean ok;
    send colaBanco(id);
    receive respuestas[id](ok);
}
```

```

process Empleado[id=0 to 1] {
    int idC;
    while(true) {
        receive colaBanco(idC);
        // atendiendo cliente
        send respuestas[idC](true);
    }
}

```

c_

```

chan colaBanco(int);
chan respuestas[N](boolean);
chan siguiente[2](texto);
chan liberado(int);

process Cliente [id=0 to n-1] {
    boolean ok;
    send colaBanco(id);
    receive respuestas[id](ok);
}

process Empleado[id=0 to 1] {
    int idC;
    while(true) {
        send liberado(id);
        receive siguiente[id](idC);
        if (idC <> -1) {
            // atender cliente
            send respuestas[idC](true);
        } else {
            delay(900); // Realiza tareas admin por 15 minutos
        }
    }
}

process Coordinador {
    int idE;

```

```

int idC;
while (true) {
    receive liberado(idE);
    if (empty(colaBanco)) {
        idC = -1;
    } else {
        receive colaBanco(idC);
    }
    send siguiente[idE] (idC);
}
}

```

Ejercicio 2

```

chan colaBanco(int);
chan respuestas[N](boolean);
chan siguiente[2](texto);
chan pedido(int);
C = 5

proccess Cliente [id=0 to n-1] {
    boolean ok;
    send AdminPedidoDireccion(id)
    send atencion()
    receive AdminRecibirDireccion[id](idE)

    pago = armarSolicitud()
    send pedido[idE](id, pago)
    receive respuestas[id](comprobante);
}

proccess Empleado[id=0 to C-1] {
    int idC;
    while(true) {
        receive pedido(idC, pago);
        comprobante = generarcomprobante(pago)
    }
}

```

```

        send respuestas[idC](true);

        send AdminFinAtencion(id)
        send atencion()
    }
}

process Admin {
    int idE;
    int idC;
    int cantxFilas[] = [(C) 0]
    while (true) {

        receive atencion()
        if (!empty(AdminPedidoDireccion)) → {
            receive AdminPedidoDireccion(idC);
            idE = buscarMinimo(cantxFilas)
            cantxFilas[idE]++
            send AdminRecibirDireccion[idC](idE)
        }
        [] (!empty(AdminFinAtencion)) → {
            receive AdminFinAtencion(idE)
            cantxFilas[idE]--
        }
    }
}

```

Ejercicio 3

```

chan colaClientes(int);
chan pedidos[N](Pedido);
chan liberado

process Cliente [id = 0 to C-1] {
    boolean ok;
    Pedido p = realizarPedido();
    send colaClientes(id, p);
}

```

```

    receive pedidos[id](p);
}

process Vendedor (id = 0 to 2){
    int idC;
    Pedido p;
    while true {
        send liberado(id);
        receive siguiente[id](idC, p)
        if (idC <> -1) {
            send pendientes(id, p);
        } else {
            delay(Random(60, 180));
        }
    }
}

process Cocinero (id = 0 to 1){
    int id;
    Pedido p;
    while true {
        receive pendientes(id, p);
        p = PrepararPedido(p);
        send pedidos[id](p)
    }
}

process Coordinador {
    int idV;
    int idC;
    Pedido p;
    while (true) {
        receive liberado(idV);
        if (empty(colaClientes)) {
            idC = -1;
        } else {
            receive colaClientes(idC, p);
        }
    }
}

```

```
        send siguiente[idV](idC, p);
    }
}
```

Ejercicio 4

```
process Cliente (id = 0 to N-1){

process Empleado {

}
```

Ejercicio 5

a_

```
chan enviarDocumento(Doc);

process Administrativo (id = 0 to N-1) {
    Doc doc;
    while true {
        doc = CrearDoc();
        send enviarDocumento(doc);
    }
}

process Impresora (id = 0 to 2) {
    Doc doc;
    while true {
        receive enviarDocumento(doc);
        ImprimirDoc(doc);
    }
}
```

```
}  
}
```

b_

```
chan enviarDocumento(Doc);  
chan enviarDocumentoDirector(Doc);  
chan atencion();  
  
process Administrativo (id = 0 to N-1) {  
    Doc doc;  
    while true {  
        doc = CrearDoc();  
        if (esDirector)  
            send enviarDocumentoDirector(doc);  
        else  
            send enviarDocumento(doc);  
    }  
    send atencion();  
}  
  
process Impresora (id = 0 to 2) {  
    Doc doc;  
    while true {  
        receive atencion();  
        if (!empty(enviarDocumentoDirector)) {  
            receive enviarDocumentoDirector(doc);  
        } else {  
            receive enviarDocumento(doc);  
        }  
        ImprimirDoc(doc);  
    }  
}  
  
process Director {  
    Doc doc;  
    while true {
```

```

    doc = CrearDoc();
    send enviarDocumentoDirector(doc);
    send atencion();
  }
}

```

c_

PMS

Ejercicio 1

```

process Examinador [int id = 0 to R-1]{
  Sitio sitio;
  while (true){
    sitio = buscarSitio();
    Admin!sitioExaminado(sitio);
  }
}

process Analizador{
  Sitio sitio;
  while (true){
    Admin!analizarSitio();
    Admin?analizarSitio(sitio);
    //chequeando sitio
  }
}

process Administrador{
  cola sitios;
  Sitio sitio
  while (true){
    do Examinador?sitioExaminado(sitio) → push (sitios, sitio);

```



```

    [] empty(sitios); Analizador?analizarSitio() → {
        pop(sitios, sitio);
        Analizador!analizarSitio(sitio);
    }
od
}
}

```

Ejercicio 2

```

process Preparador {

    Muestra m;
    while true {
        m = PrepararMuestra();
        Buffer!envioMuestra(m);
    }
}

process Buffer {
    cola muestras
    while true {
        do Preparador?envioMuestra(m); →
            push(muestras, m);
        [] !empty(muestras), Buffer?pedido(m) →
            pop(m, muestras)
            ArmadorDeSet!envioMuestra(m)

    }
}

process ArmadorDeSet {
    Muestra m;
    Set s;
    Resultado r;
    while true {

```

```

    Buffer!pedido(m);
    Buffer?envioMuestra(m);
    s = PrepararSet(m);
    Analizador!envioSet(s);
    Analizador?envioResultado(r);
  }
}

process Analizador {
  Resultado r;
  while true {
    ArmadorDeSet?envioSet(s);
    r = Analizar(s);
    ArmadorDeSet!envioResultado(r);
  }
}

```

Ejercicio 3

a_

```

process Alumno [int id = 0 to n-1]{
  Examen examen;
  int nota;
  examen = resolverExamen();
  Admin!examenRealizado(examen, id);
  Profesor?esperarNota(nota);
}

process Profesor {
  Examen examen;
  int nota;
  for (int i=0; i<n; i++) {
    Admin!pedirExamen();
    Admin?corregirExamen(examen, id);
    nota = random(10 - 1)
    Alumno[idA]!esperarNota(nota);
  }
}

```

```

    }
}

process Administrador{
    cola examen;
    int idA;
    Examen examen
    int cantExamenes = 0
    do cantExamenes < N; Alumno[*]?examenRealizado(examen, idA) → {
        push (examen, (examen, idA));
        cantExamenes++;
    }
    [] !empty(examen); Profesor?pedirExamen() → {
        pop(examen, (examen, idA));
        Profesor!corregirExamen(examen, idA);
    }
od
}

```

b_

```

process Alumno [int id = 0 to n-1]{
    Examen examen;
    int nota;
    examen = resolverExamen();
    Admin!examenRealizado(examen, id);
    Profesor[*]?esperarNota(nota);
}

process Profesor [int id = 0 to p-1] {
    Examen examen;
    int nota, idA;
    Admin!pedirExamen(id);
    Admin?corregirExamen(examen, idA);
    while(examen != null) {
        nota = random(10 - 1)
        Alumno[idA]!esperarNota(nota);
        Admin!pedirExamen(id);
    }
}

```

```

        Admin?corregirExamen(examen, idA);
    }
}

process Admin{
    cola examenenes;
    int idA, idP;
    int cantAlumnos = 0;
    Examen examen;
    do cantAlumnos < n; Alumno[*]?examenRealizado(examen, idA) → {
        push (examenenes, (examen, idA));
        cantAlumnos++;
    }
    [] !empty(examenenes); Profesor[*]?pedirExamen(idP) → {
        pop(examen, (examen, idA);
        Profesor[idP]!corregirExamen(examen, idA);
    }
od
}

for (int i=0; i<p; i++) {
    Profesor[i]!corregirExamen(null, 0);
}
}

```

c_

```

process Alumno [int id = 0 to n-1]{
    Examen examen;
    int nota;
    Admin!!legue();
    Admin!arrancar();
    examen = resolverExamen();
    Admin!examenRealizado(examen, id);
    Profesor?esperarNota(nota);
}

process Profesor [int id = 0 to p-1] {

```

```

Examen examen;
int nota, idA;
Admin!pedirExamen(id);
Admin?corregirExamen(examen, idA);
while(examen != null) {
    nota = random(10 - 1)
    Alumno[idA]!esperarNota(nota);
    Admin!pedirExamen(id);
    Admin?corregirExamen(examen, idA);
}
}

process Admin{
    cola examenenes;
    int idA, idP;
    int cantAlumnos = 0;
    Examen examen;

    for (int i = 0 to n-1) {
        Alumno[*]?llegue();
    }

    for (int i = 0 to n-1) {
        Alumno[*]?arranca();
    }

    do cantAlumnos < n; Alumno[*]?examenRealizado(examen, idA) → {
        push (examenenes, (examen, idA));
        cantAlumnos++;
    }
    [] !empty(examenenes); Profesor[*]?pedirExamen(idP) → {
        pop(examen, (examen, idA);
        Profesor[idP]!corregirExamen(examen, idA);
    }
od
}

for (int i=0; i<p; i++) {

```

```

        Profesor[i]!corregirExamen(null, 0);
    }
}

```

Ejercicio 4

a_

```

process Persona[id = 0 to P] {
    Empleado!SolicitarUso(id);
    Empleado?Permiso();
    // Lo usa
    Empleado!Termine();
}

process Empleado {
    int idP;
    for (int i = 0 to P-1) {
        Persona[*]?SolicitarUso(idP);
        Persona[idP]!Permiso();
        Persona[idP]?Termine();
    }
}

```

b_

```

process Persona[id = 0 to P] {
    Empleado!SolicitarUso(id);
    Empleado?Permiso();
    // Lo usa
    Empleado!Termine();
}

process Empleado {
    // Uso la variable i como si fuera siguiente.
}

```

```

int idP;
for (int i = 0 to P-1) {
    Persona[i]?SolicitarUso(idP);
    Persona[idP]!Permiso();
    Persona[idP]?Termine();
}
}

```

c_

```

process Persona[id = 0 to P] {
    Admin!SolicitarUso(id);
    Empleado?Permiso();
    // Lo usa
    Empleado!Termine();
}

process Admin {
    cola colaPersonas;
    int cant = 0;

    do cant < P; Persona[*]?SolicitarUso(idP) → {
        push(colaPersonas, idP);
        cant++;
    }
    [] !empty(colaPersonas); Empleado?libre() → {
        pop(colaPersonas, idP);
        Empleado!Permiso(idP);
    }
    od
}

process Empleado {
    for (int i = 0 to P-1) {
        Admin!libre();
        Admin?permiso(idP);
        Persona[idP]!permiso();
        Persona[idP]?Termine();
    }
}

```

```
}  
}
```

Ejercicio 5

```
process Espectador [id = 0 to E-1] {  
    Admin!SolicitarUso(id);  
    Admin?UsarMaquina();  
    // La usa  
    Expendedora!Termine();  
}  
  
process Admin {  
    int idE;  
    int cant = 0;  
    cola cola;  
    do cant < E; Espectador[*]?SolicitarUso(idE) → {  
        push(cola, idE);  
        cant++;  
    }  
    [] !empty(cola); Expendedora?Libre() → {  
        pop(cola, idE);  
        Espectador[idE]!UsarMaquina();  
    }  
    od  
}  
  
process Expendedora {  
    for (int i = 0 to E-1) {  
        Admin!libre();  
        Espectador[*]?Termine();  
    }  
}
```


